Assignment - Module 5

Meenu S

March 27, 2026

Introduction

The study of **eigenvalues** and **eigenvectors** is central to understanding linear transformations and their geometric properties. For a square matrix A, an eigenvector v and its corresponding eigenvalue λ satisfy the characteristic equation:

$$Av = \lambda v \tag{1}$$

This assignment explores the systematic process of shifting from standard basis representations to the eigen-basis, a process known as **diagonalization**.

We specifically investigate **symmetric matrices**, which occupy a unique position in linear algebra due to the Spectral Theorem. Unlike general matrices, symmetric matrices are guaranteed to be **orthogonally diagonalizable**, allowing for the decomposition:

$$A = QDQ^T \tag{2}$$

where Q is an orthogonal matrix of eigenvectors and D is a diagonal matrix of eigenvalues. Finally, we will discuss the practical **applications** of these concepts.

1 Eigenvalues and Eigenvectors

In linear algebra, a linear transformation can often be understood by identifying vectors whose direction remains unchanged by the transformation. These vectors are known as eigenvectors, and the scaling factor by which they are stretched or compressed is the eigenvalue.

1.1 Fundamental Concepts

Let A be an $n \times n$ square matrix over a field $\mathbb{F}$ (typically $\mathbb{R}$ or $\mathbb{C}$). A non-zero vector $\mathbf{v} \in \mathbb{F}^n$ is called an **eigenvector** of A if there exists a scalar $\lambda \in \mathbb{F}$, called an **eigenvalue**, such that:

$$A\mathbf{v} = \lambda\mathbf{v} \tag{3}$$

This implies that the action of the matrix A on the vector $\mathbf{v}$ is equivalent to simple scalar multiplication.

1.2 The Characteristic Equation

A scalar λ is an eigenvalue of an $n \times n$ matrix A if and only if $\det(A - \lambda I) = 0$.

By definition, λ is an eigenvalue if there exists a non-zero vector $\mathbf{v}$ such that $A\mathbf{v} = \lambda\mathbf{v}$. Rearranging this gives:

$$A\mathbf{v} - \lambda I\mathbf{v} = \mathbf{0}$$
$$(A - \lambda I)\mathbf{v} = \mathbf{0}$$

This is a homogeneous linear system. For a non-trivial solution ($\mathbf{v} \neq \mathbf{0}$) to exist, the coefficient matrix $(A - \lambda I)$ must be non-invertible. From the properties of determinants, a square matrix is non-invertible if and only if its determinant is zero. Thus, $\det(A - \lambda I) = 0$.

1.3 Properties

The eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$ of a matrix A relate directly to its fundamental invariants:

- $\sum_{i=1}^{n} \lambda_i = \text{tr}(A)$ (The sum of eigenvalues equals the trace).
- $\prod_{i=1}^{n} \lambda_i = \det(A)$ (The product of eigenvalues equals the determinant).

Let $p(\lambda) = \det(A - \lambda I)$ be the characteristic polynomial. According to the Fundamental Theorem of Algebra, $p(\lambda)$ can be factored into n linear factors:

$$p(\lambda) = (\lambda_1 - \lambda)(\lambda_2 - \lambda)\ldots(\lambda_n - \lambda) \tag{4}$$

The product of the eigenvalues $\prod_{i=1}^{n} \lambda_i$ is equal to $\det(A)$.

Evaluate the characteristic polynomial at $\lambda = 0$:

- From the determinant definition: $p(0) = \det(A - 0I) = \det(A)$.
- From the factored form: $p(0) = (\lambda_1 - 0)(\lambda_2 - 0)\ldots(\lambda_n - 0) = \lambda_1\lambda_2\ldots\lambda_n$.

Equating the two yields $\det(A) = \prod_{i=1}^{n} \lambda_i$.

The identity $\text{tr}(A) = \sum_{i=1}^{n} \lambda_i$ is a consequence of two deep mathematical properties: the structure of the characteristic polynomial and the cyclic property of the trace.

The characteristic polynomial of an $n \times n$ matrix A is defined as $p(\lambda) = \det(A - \lambda I)$. By the Leibniz formula for determinants:

$$\det(A - \lambda I) = \prod_{i=1}^{n}(a_{ii} - \lambda) + \text{lower-degree terms in } \lambda \tag{5}$$

When expanding the product $\prod_{i=1}^{n}(a_{ii} - \lambda)$, we look for the coefficient of $(-\lambda)^{n-1}$. To obtain λ^{n-1}, we must choose the term $(-\lambda)$ from $(n-1)$ factors and the constant a_{ii} from exactly one factor. Summing all such possible combinations gives:

$$(a_{11} + a_{22} + \cdots + a_{nn})(-\lambda)^{n-1} = \text{tr}(A)(-\lambda)^{n-1} \tag{6}$$

Similarly, if we factor $p(\lambda)$ using its roots λ_i, we get $p(\lambda) = \prod_{i=1}^{n}(\lambda_i - \lambda)$. Expanding this, the coefficient of $(-\lambda)^{n-1}$ is $(\lambda_1 + \lambda_2 + \cdots + \lambda_n)$. Since both expressions describe the same polynomial, their coefficients must be equal:

$$\operatorname{tr}(A) = \sum_{i=1}^{n} \lambda_i \tag{7}$$

A more abstract but powerful proof relies on the fact that the trace is a **similarity invariant**.

Cyclic Property: For any two matrices A and B, $\operatorname{tr}(AB) = \operatorname{tr}(BA)$.

By definition, $(AB)_{ii} = \sum_j a_{ij}b_{ji}$. Thus:

$$\operatorname{tr}(AB) = \sum_i \sum_j a_{ij}b_{ji} = \sum_j \sum_i b_{ji}a_{ij} = \operatorname{tr}(BA)$$

If A and B are similar ($B = P^{-1}AP$), then $\operatorname{tr}(A) = \operatorname{tr}(B)$.

Using the cyclic property $\operatorname{tr}(XY) = \operatorname{tr}(YX)$, let $X = P^{-1}$ and $Y = AP$:

$$\operatorname{tr}(B) = \operatorname{tr}(P^{-1}(AP)) = \operatorname{tr}((AP)P^{-1}) = \operatorname{tr}(A(PP^{-1})) = \operatorname{tr}(AI) = \operatorname{tr}(A)$$

If A is diagonalizable, there exists a similarity transformation such that $P^{-1}AP = D$, where D is a diagonal matrix of eigenvalues. Since $\operatorname{tr}(A) = \operatorname{tr}(D)$ and the trace of a diagonal matrix is simply the sum of its diagonal entries, it follows that $\operatorname{tr}(A) = \sum \lambda_i$.

1.4 Linear Independence of Eigenvectors

Eigenvectors corresponding to distinct eigenvalues are linearly independent.

Let $\mathbf{v}_1, \ldots, \mathbf{v}_k$ be eigenvectors for distinct eigenvalues $\lambda_1, \ldots, \lambda_k$. We use induction on k. For $k = 1$, $\mathbf{v}_1 \neq \mathbf{0}$ is independent. Assume the set $\{\mathbf{v}_1, \ldots, \mathbf{v}_{m-1}\}$ is independent but $\{\mathbf{v}_1, \ldots, \mathbf{v}_m\}$ is dependent. Then:

$$c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_m\mathbf{v}_m = \mathbf{0} \quad \text{(where some } c_i \neq 0) \tag{*}$$

Multiply (*) by A: $c_1\lambda_1\mathbf{v}_1 + \cdots + c_m\lambda_m\mathbf{v}_m = \mathbf{0}$. Multiply (*) by λ_m: $c_1\lambda_m\mathbf{v}_1 + \cdots + c_m\lambda_m\mathbf{v}_m = \mathbf{0}$. Subtracting the two equations: $c_1(\lambda_1 - \lambda_m)\mathbf{v}_1 + \cdots + c_{m-1}(\lambda_{m-1} - \lambda_m)\mathbf{v}_{m-1} = \mathbf{0}$. Since the first $m-1$ vectors are independent and $\lambda_i \neq \lambda_m$, all c_i must be zero, contradicting the dependence assumption.

1.5 Algebraic and Geometric Multiplicities

For a square matrix A, every eigenvalue λ is associated with two distinct types of multiplicity which determine the diagonalizability of the matrix.

Algebraic Multiplicity (AM): The algebraic multiplicity of an eigenvalue λ_i, denoted $AM(\lambda_i)$, is the multiplicity of λ_i as a root of the characteristic polynomial $p(z) = \det(A - zI)$.

Geometric Multiplicity (GM): The geometric multiplicity of an eigenvalue λ_i, denoted $GM(\lambda_i)$, is the dimension of the corresponding eigenspace $E_{\lambda_i} = \text{null}(A - \lambda_i I)$. It represents the maximum number of linearly independent eigenvectors associated with λ_i.

The Multiplicity Inequality: For every eigenvalue λ_i of a matrix A:

$$1 \leq GM(\lambda_i) \leq AM(\lambda_i) \tag{8}$$

Consider the matrix $B = \begin{pmatrix} 3 & 1 \\ 0 & 3 \end{pmatrix}$.

Step 1: Calculate AM. The characteristic equation is $\det(B - \lambda I) = (3 - \lambda)^2 = 0$. The eigenvalue $\lambda = 3$ has an algebraic multiplicity of $AM = 2$.

Step 2: Calculate GM. We solve $(B - 3I)\mathbf{v} = \mathbf{0}$:

$$\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \implies y = 0$$

The only independent eigenvector is $\mathbf{v} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$. Thus, the dimension of the eigenspace is $GM = 1$.

Since $GM < AM$, the matrix B is defective and cannot be diagonalized.

Condition for Diagonalization: A matrix A is diagonalizable if and only if $GM(\lambda_i) = AM(\lambda_i)$ for all eigenvalues λ_i of A.

1.6 Geometric Interpretation

A linear transformation $T(\mathbf{v}) = A\mathbf{v}$ typically alters both the magnitude and the direction of a vector. Geometrically, eigenvectors represent the **invariant lines** of the transformation.

An eigenvector $\mathbf{v}$ defines a line (a one-dimensional subspace) through the origin. When A is applied to any vector on this line, the resulting vector $A\mathbf{v}$ remains on the same line. The eigenvalue λ determines how the transformation behaves along this axis:

- **Stretching/Compression:** If $|\lambda| > 1$, the transformation stretches space along the eigenvector. If $|\lambda| < 1$, it compresses space.
- **Inversion:** If $\lambda < 0$, the transformation reflects space through the origin along that axis.
- **Singularity:** If $\lambda = 0$, the entire dimension associated with that eigenvector is collapsed into the origin.

1.7 The Characteristic Geometry of Matrices

The set of all eigenvectors for a specific eigenvalue λ forms the **eigenspace**, denoted E_λ.

- If a 2×2 matrix has two distinct real eigenvalues, it stretches space along two independent axes.

- If a matrix has complex eigenvalues, it indicates a **rotational component**, as no real line remains invariant under the transformation.
- The determinant, $\det(A) = \prod \lambda_i$, represents the total change in **area** (2D) or **volume** (3D) resulting from these combined directional stretches.

2 Diagonalization

2.1 Diagonalizability

A matrix $A \in \mathbb{R}^{n \times n}$ is diagonalizable if it is similar to a diagonal matrix D. This occurs if and only if A possesses n linearly independent eigenvectors. In this case:

$$A = PDP^{-1} \tag{9}$$

where the columns of P are the eigenvectors and the diagonal entries of D are the corresponding eigenvalues.

2.2 Symmetric Matrices and Orthogonal Diagonalization

Real symmetric matrices ($A = A^T$) possess unique properties that simplify their analysis. These properties are summarized by the **Spectral Theorem**.

The Real Spectral Theorem: If A is a real symmetric matrix, then:

1. All eigenvalues of A are real.
2. Eigenvectors belonging to distinct eigenvalues are orthogonal.
3. A is orthogonally diagonalizable.

Let $\mathbf{v}_1, \mathbf{v}_2$ be eigenvectors with distinct eigenvalues λ_1, λ_2. Consider the quantity $\mathbf{v}_1^T A \mathbf{v}_2$:

$$\mathbf{v}_1^T (A\mathbf{v}_2) = \mathbf{v}_1^T (\lambda_2 \mathbf{v}_2) = \lambda_2 (\mathbf{v}_1 \cdot \mathbf{v}_2)$$

Using the symmetry of A ($A = A^T$):

$$(A\mathbf{v}_1)^T \mathbf{v}_2 = (\lambda_1 \mathbf{v}_1)^T \mathbf{v}_2 = \lambda_1 (\mathbf{v}_1 \cdot \mathbf{v}_2)$$

Equating the two: $\lambda_1 (\mathbf{v}_1 \cdot \mathbf{v}_2) = \lambda_2 (\mathbf{v}_1 \cdot \mathbf{v}_2)$. Since $\lambda_1 \neq \lambda_2$, the only solution is $(\mathbf{v}_1 \cdot \mathbf{v}_2) = 0$, proving the vectors are orthogonal.

2.3 Consequences of the Spectral Theorem

Based on the proof of orthogonality, we can derive the following functional properties for symmetric matrices:

2.3.1 1. Orthonormal Eigenbasis

Since eigenvectors $\mathbf{v}_i$ corresponding to distinct eigenvalues are orthogonal, we can normalize each vector such that $\mathbf{u}_i = \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|}$. The resulting set $\{\mathbf{u}_1, \ldots, \mathbf{u}_n\}$ forms an **orthonormal basis** for $\mathbb{R}^n$.

2.3.2 2. The Matrix Form: $A = Q\Lambda Q^T$

If we construct an orthogonal matrix Q using these orthonormal eigenvectors as columns, the diagonalizing relation $A = QDQ^{-1}$ simplifies significantly.

For an orthogonal matrix Q the identity $Q^T = Q^{-1}$ arises directly from the orthonormality of its columns.

The ij-th entry of the product Q^TQ is given by the dot product of the columns of Q:

$$(Q^TQ)_{ij} = \mathbf{u}_i \cdot \mathbf{u}_j$$

By the definition of an orthonormal set:

$$\mathbf{u}_i \cdot \mathbf{u}_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

This implies that $Q^TQ = I$. Multiplying both sides by Q^{-1} on the right (or simply applying the definition of an inverse), we conclude:

$$Q^T = Q^{-1} \tag{10}$$

$$A = Q\Lambda Q^T \tag{11}$$

where Λ is the diagonal matrix of real eigenvalues.

2.3.3 Case 1: Symmetric Matrix (Orthogonally Diagonalizable)

For $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, the eigenvalues are $\lambda = \{3, 1\}$. The normalized eigenvectors are:

$$\mathbf{u}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \mathbf{u}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Since $\mathbf{u}_1 \cdot \mathbf{u}_2 = 0$, the matrix $Q = [\mathbf{u}_1 \ \mathbf{u}_2]$ is orthogonal, and A can be decomposed as $Q\Lambda Q^T$.

$$Q = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix}, \quad \Lambda = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}$$

Since Q is orthogonal ($Q^T = Q^{-1}$), we verify the decomposition $A = Q\Lambda Q^T$:

$$\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} = A$$

2.3.4 Case 2: Defective Matrix (Non-Diagonalizable)

For $B = \begin{pmatrix} 2 & 1 \\ 0 & 2 \end{pmatrix}$, the eigenvalue $\lambda = 2$ has an algebraic multiplicity of 2. However, the null space of $(B - 2I)$ is spanned only by $\mathbf{v} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, giving a geometric multiplicity of 1. Because $GM < AM$, B is not diagonalizable.

3 Applications of Eigenvalues and Eigenvectors

Eigenvalues and eigenvectors are fundamental tools in linear algebra that help simplify complex systems. They reveal the intrinsic structure of matrices and are widely used in population models, differential equations, and geometry.

3.1 Application 1: Population Growth Models

3.1.1 Age Distribution Vector

A population divided into age groups can be represented as:

$$x_n = \begin{pmatrix} x_{1,n} \\ x_{2,n} \\ x_{3,n} \end{pmatrix}$$

where:

- $x_{1,n}$ = number of young individuals
- $x_{2,n}$ = number of middle-aged individuals
- $x_{3,n}$ = number of old individuals

The population evolves as:

$$x_{n+1} = Ax_n$$

where A is the **age transition matrix**.

3.1.2 Example: Rabbit Population

$$A = \begin{pmatrix} 0 & 6 & 8 \\ 0.5 & 0 & 0 \\ 0 & 0.5 & 0 \end{pmatrix}$$

The Leslie Matrix Model

The population dynamics in this study are modeled using a **Leslie Matrix**, a discrete-time age-structured model. For a population divided into n age classes, the Leslie Matrix A is defined as:

$$A = \begin{bmatrix} f_1 & f_2 & f_3 & \dots & f_n \\ s_1 & 0 & 0 & \dots & 0 \\ 0 & s_2 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & s_{n-1} & 0 \end{bmatrix}$$

where:

- f_i represents the **fecundity** (average number of offspring) of an individual in age class i.
- s_i represents the **survival probability** of an individual in age class i transitioning to age class $i + 1$.

The population vector at time $t + 1$ is calculated by the linear recurrence $\mathbf{x}_{t+1} = A\mathbf{x}_t$. In our specific model, the matrix $A = \begin{pmatrix} 0 & 6 & 8 \\ 0.5 & 0 & 0 \\ 0 & 0.5 & 0 \end{pmatrix}$ indicates that only the second and third age groups are reproductive, while 50% of individuals survive to the next stage. Initial population:

$$x_1 = \begin{pmatrix} 24 \\ 24 \\ 20 \end{pmatrix}$$

Next year:

$$x_2 = Ax_1 = \begin{pmatrix} 304 \\ 12 \\ 12 \end{pmatrix}$$

Interpretation:

- First row → births
- Second row → survival to next age
- Third row → aging

3.1.3 Stable Age Distribution

A stable population satisfies:

$$Ax = \lambda x$$

Thus:

- x = eigenvector (stable ratio)
- λ = growth factor

Example:

$$x = \begin{pmatrix} 16t \\ 4t \\ t \end{pmatrix} \Rightarrow 16 : 4 : 1$$

Important Notes:

- $\lambda > 1 \rightarrow$ growth
- $0 < \lambda < 1 \rightarrow$ decay
- $\lambda < 0 \rightarrow$ not physically meaningful (negative population)

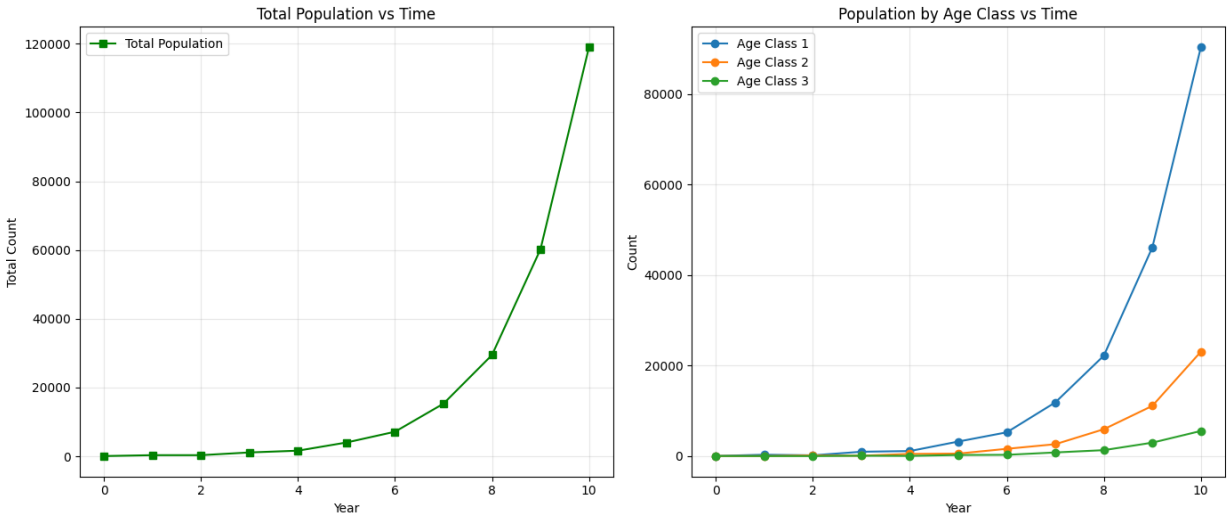
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# 1. Define the matrix A and the initial population vector x0
A = np.array([[0, 6,   8],
              [0.5, 0,  0],
              [0,   0.5, 0]])

x0 = np.array([24, 24, 20])

# 2. Compute the population vectors for the next 10 years
years = 10
pop_history = [x0]
current_x = x0.astype(float)

for i in range(years):
    current_x = A @ current_x
    pop_history.append(current_x)

# 3. Display the age distribution at each year in a clear tabular form
df = pd.DataFrame(pop_history, columns=['Age Class 1', 'Age Class 2', 'Age Class
    3'])
df.index.name = 'Year'
print(" Age Distribution Table ---")
print(df.round(2))
print("-" * 40)

# 4. Compute the eigenvalues and eigenvectors of the matrix A
eigenvalues, eigenvectors = np.linalg.eig(A)
print("\nEigenvalues and Eigenvectors ---")
```

```
print(f"Eigenvalues: {eigenvalues.round(4)}")

# 5. Identify the dominant eigenvalue and the corresponding eigenvector
# The dominant eigenvalue has the largest absolute magnitude
dom_idx = np.argmax(np.abs(eigenvalues))
dom_lambda = eigenvalues[dom_idx].real
dom_vec = eigenvectors[:, dom_idx].real
print(f"Dominant Eigenvalue (λ): {dom_lambda:.4f}")

# 6. Normalize the dominant eigenvector so entries sum to 1 (proportions)
stable_dist = dom_vec / np.sum(dom_vec)
print(f"Normalized Dominant Eigenvector: {stable_dist.round(4)}")

# 7. Compare long-term population ratios with the stable age distribution
year_10_total = np.sum(pop_history[-1])
year_10_ratio = pop_history[-1] / year_10_total
print(f"\nComparison ---")
print(f"Year 10 Ratios (Iteration): {year_10_ratio.round(4)}")
print(f"Stable Distribution (Eigen): {stable_dist.round(4)}")

# 8. Plot: Total population vs time and each age class vs time
time_steps = np.arange(0, years + 1)
total_pop = [np.sum(p) for p in pop_history]

plt.figure(figsize=(14, 6))

# Plot: Total Population vs Time
plt.subplot(1, 2, 1)
plt.plot(time_steps, total_pop, 's-', color='green', label='Total Population')
plt.title('Total Population vs Time')
plt.xlabel('Year')
plt.ylabel('Total Count')
plt.grid(True, alpha=0.3)
plt.legend()

# Plot: Age Classes vs Time
plt.subplot(1, 2, 2)
plt.plot(time_steps, df.iloc[:, 0], 'o-', label='Age Class 1')
plt.plot(time_steps, df.iloc[:, 1], 'o-', label='Age Class 2')
plt.plot(time_steps, df.iloc[:, 2], 'o-', label='Age Class 3')
plt.title('Population by Age Class vs Time')
plt.xlabel('Year')
plt.ylabel('Count')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
```

```
plt.show()
```

OUTPUT

```
Age Distribution Table ---
      Age Class 1 Age Class 2 Age Class 3
Year
0            24.0        24.0        20.0
1           304.0        12.0        12.0
2           168.0       152.0         6.0
3           960.0        84.0        76.0
4          1112.0       480.0        42.0
5          3216.0       556.0       240.0
6          5256.0      1608.0       278.0
7         11872.0      2628.0       804.0
8         22200.0      5936.0      1314.0
9         46128.0     11100.0      2968.0
10        90344.0     23064.0      5550.0
----------------------------------------

Eigenvalues and Eigenvectors ---
Eigenvalues: [ 2. -1. -1.]
Dominant Eigenvalue (λ): 2.0000
Normalized Dominant Eigenvector: [0.7619 0.1905 0.0476]

Comparison ---
Year 10 Ratios (Iteration): [0.7595 0.1939 0.0467]
Stable Distribution (Eigen): [0.7619 0.1905 0.0476]
```



The dominant eigenvalue represents the long-term growth rate of the population. Since it is greater than 1, the total population increases approximately exponentially over time.

The corresponding eigenvector represents the stable age distribution. After normalization, it gives the fixed proportions of individuals in each age class. In this model, the ratio approaches $16 : 4 : 1$, meaning most individuals are in the youngest age class.

From the computed iterations, it is observed that although the total population changes, the relative proportions of the age groups gradually stabilize and match the eigenvector.

Thus, the dominant eigenvalue determines how fast the population grows, while the dominant eigenvector determines how the population is distributed among age classes in the long run.

3.2 Application 2: Systems of Differential Equations

3.2.1 Matrix Form

A system can be written as:

$$y' = Ay$$

where:

$$y = \begin{pmatrix} y_1(t) \\ y_2(t) \end{pmatrix}$$

3.2.2 Key Idea

We try a solution of the form:

$$y(t) = ve^{\lambda t}$$

Substituting into $y' = Ay$:

$$\lambda v e^{\lambda t} = Ave^{\lambda t}$$

Cancelling $e^{\lambda t}$:

$$Av = \lambda v$$

Thus eigenvalues naturally arise.

3.2.3 Example

$$A = \begin{pmatrix} 3 & 2 \\ 6 & 1 \end{pmatrix}$$

Characteristic equation:

$$\lambda^2 - 4\lambda - 9 = 0$$

$$\lambda = 2 \pm \sqrt{13}$$

Eigenvectors:

$$v_1 = \begin{pmatrix} 1 \\ \frac{-1+\sqrt{13}}{2} \end{pmatrix}, \quad v_2 = \begin{pmatrix} 1 \\ \frac{-1-\sqrt{13}}{2} \end{pmatrix}$$

General solution:

$$y(t) = C_1 v_1 e^{\lambda_1 t} + C_2 v_2 e^{\lambda_2 t}$$

Important Concepts:

- $y_1(t), y_2(t)$ = components of vector
- $y^{(1)}, y^{(2)}$ = different solutions (NOT derivatives)
- Eigenvectors = directions of motion
- Eigenvalues = growth/decay rates

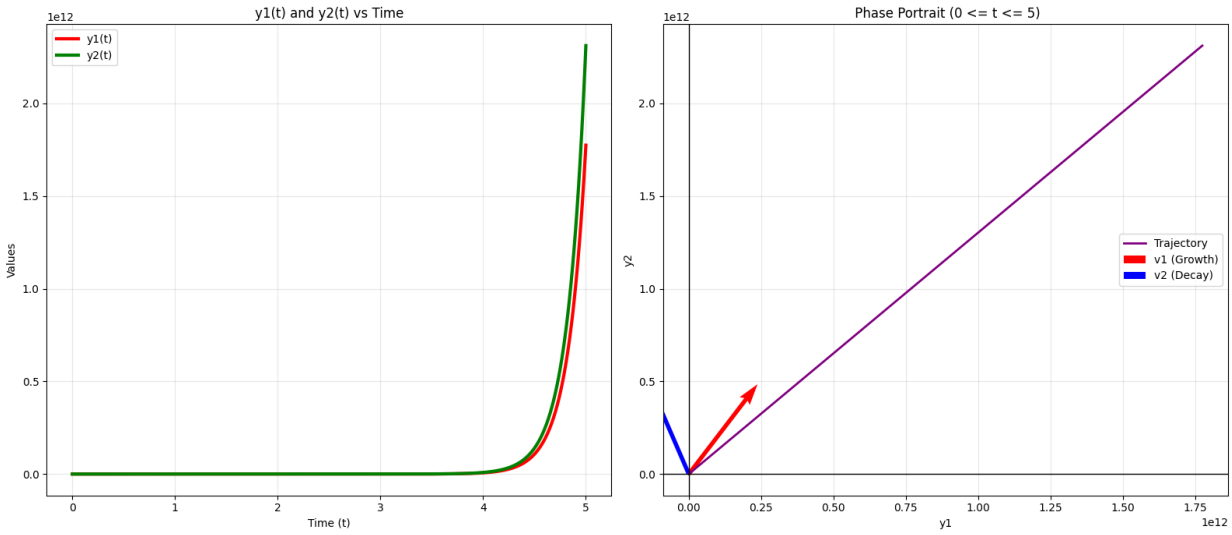
```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

# 1. Define the matrix A
A = np.array([[3, 2],
              [6, 1]])

# 2. Compute the exact eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(A)
lam1, lam2 = eigenvalues
v1 = eigenvectors[:, 0]
v2 = eigenvectors[:, 1]

print(" Eigen-Analysis ---")
print(f"Eigenvalue 1 (L1): {lam1:.4f}")
print(f"Eigenvector 1 (v1): {v1}")
print(f"Eigenvalue 2 (L2): {lam2:.4f}")
print(f"Eigenvector 2 (v2): {v2}")

# 3. Verify diagonalization: P^-1 * A * P = D
P = eigenvectors
P_inv = np.linalg.inv(P)
# Cleaning near-zero values to 0 for the report
D = np.round(P_inv @ A @ P, 10)

print("\n Diagonal Matrix D (Cleaned) ---")
print(D)
```

```
# 5. Initial Condition y(0) = [1, 2] to find constants c1 and c2
y0 = np.array([1, 2])
c = np.linalg.solve(P, y0)
c1, c2 = c
print("c1=",c1,"c2=",c2)
# --- TASK 4: PRINTING THE GENERAL SOLUTION SYMBOLICALLY ---
print("\n General Solution from Diagonalization ---")
solution_str = (f"y(t) = ({c1:.4f}) * e^({lam1:.4f}t) * {v1.round(4)} + "
                f"({c2:.4f}) * e^({lam2:.4f}t) * {v2.round(4)}")
print(solution_str)

# 6. Compute numerical solution for 0 <= t <= 5
t_steps = np.linspace(0, 5, 11)

def system_func(y, t, A_mat):
    return A_mat @ y

y_numeric_table = odeint(system_func, y0, t_steps, args=(A,))

print("\n Numerical Solution Table ---")
print(f"{'t':<6} {'y1(t)':<20} {'y2(t)':<20}")
print("-" * 50)
for i in range(len(t_steps)):
    print(f"{t_steps[i]:<6.2f} {y_numeric_table[i, 0]:<20.4f} {y_numeric_table[i,
        1]:<20.4f}")

# 7. Plotting
t_plot = np.linspace(0, 5, 1000)
# Formula: y(t) = c1*exp(L1*t)*v1 + c2*exp(L2*t)*v2
y_symbolic_plot = np.array([c1 * np.exp(lam1 * val) * v1 + c2 * np.exp(lam2 *
    val) * v2 for val in t_plot])

plt.figure(figsize=(16, 7))

# Subplot 1: Time Series
plt.subplot(1, 2, 1)
plt.plot(t_plot, y_symbolic_plot[:, 0], 'r-', lw=3, label='y1(t)')
plt.plot(t_plot, y_symbolic_plot[:, 1], 'g-', lw=3, label='y2(t)')
plt.title('y1(t) and y2(t) vs Time')
plt.xlabel('Time (t)')
plt.ylabel('Values')
plt.grid(True, alpha=0.3)
plt.legend()

# Subplot 2: Phase Portrait
plt.subplot(1, 2, 2)
plt.plot(y_symbolic_plot[:, 0], y_symbolic_plot[:, 1], 'purple', lw=2,
```

```
    label='Trajectory')
plt.quiver(0, 0, v1[0], v1[1], color='red', scale=5, label='v1 (Growth)')
plt.quiver(0, 0, v2[0], v2[1], color='blue', scale=5, label='v2 (Decay)')
plt.axhline(0, color='black', lw=1); plt.axvline(0, color='black', lw=1)
plt.title(' Phase Portrait (0 <= t <= 5)')
plt.xlabel('y1')
plt.ylabel('y2')
plt.grid(True, alpha=0.3)
plt.legend()

plt.tight_layout()
plt.show()
```

OUTPUT

```
Eigen-Analysis ---
Eigenvalue 1 (L1): 5.6056
Eigenvector 1 (v1): [0.60889368 0.79325185]
Eigenvalue 2 (L2): -1.6056
Eigenvector 2 (v2): [-0.3983218 0.91724574]

 Diagonal Matrix D (Cleaned) ---
[[ 5.60555128 -0.      ]
 [-0.        -1.60555128]]
c1= 1.9599074407209982 c2= 0.4854749228637292

 General Solution from Diagonalization ---
y(t) = (1.9599) * e^(5.6056t) * [0.6089 0.7933] + (0.4855) * e^(-1.6056t) *
    [-0.3983 0.9172]

  Numerical Solution Table ---
t     y1(t)              y2(t)
-------------------------------------------------
0.00  1.0000             2.0000
0.50  19.5925            25.8371
1.00  324.4779           422.8619
1.50  5351.3785          6971.7083
2.00  88246.4123         114965.3042
2.50  1455214.8286       1895818.4390
3.00  23997009.7678      31262719.7095
3.50  395719220.2036     515533359.4660
4.00  6525550578.9481    8501328317.0427
4.50  107608648371.1352  140189925507.1841
5.00  1774504856096.0398 2311781695559.0400
```



The system was solved using two distinct approaches to ensure the accuracy of the model:

- **Symbolic Method:** Utilizing the eigenvalues ($\lambda_1 \approx 5.6056, \lambda_2 \approx -1.6056$) and their corresponding eigenvectors, we constructed the general solution:

  $$\mathbf{y}(t) = c_1 e^{\lambda_1 t}\mathbf{v}_1 + c_2 e^{\lambda_2 t}\mathbf{v}_2$$

  For the initial condition $\mathbf{y}(0) = [1, 2]^T$, the constants were determined to be $c_1 \approx 1.9599$ and $c_2 \approx 0.4855$.

- **Numerical Method:** The system was integrated using Python's `scipy.integrate.odeint` over the interval $t \in [0, 5]$.

- **Comparison:** The numerical results (e.g., $y_1(5) \approx 1.77 \times 10^{12}$) perfectly match the values produced by the symbolic growth model. The phase portrait confirms that the numerical trajectory follows the exact path dictated by the analytical eigenvectors, demonstrating that the discrete approximation converges to the theoretical solution.

The behavior of the linear system is entirely governed by the properties of the matrix A. The eigenvalues represent the *characteristic exponents* of the system:

- **Positive Eigenvalue ($\lambda_1 > 0$):** Since $\lambda_1 \approx 5.6056$, the term $e^{5.6056t}$ grows exponentially. This identifies $\mathbf{v}_1$ as an unstable direction, causing the solution to diverge from the origin.

- **Negative Eigenvalue ($\lambda_2 < 0$):** Since $\lambda_2 \approx -1.6056$, the term $e^{-1.6056t}$ decays toward zero. This identifies $\mathbf{v}_2$ as a stable direction.

- **Nature of Equilibrium:** Because the eigenvalues have opposite signs, the origin $(0, 0)$ is classified as a **Saddle Point**.

The eigenvectors $\mathbf{v}_1$ and $\mathbf{v}_2$ define the *invariant manifolds* of the system:

- **The Growth Axis:** The eigenvector $\mathbf{v}_1 \approx [0.6089, 0.7933]^T$ defines the line along which the solution eventually aligns. As $t \to \infty$, the growing exponential term dominates, and the trajectory becomes parallel to this vector.
- **The Decay Axis:** The eigenvector $\mathbf{v}_2 \approx [-0.3983, 0.9172]^T$ defines the direction of initial contraction.

In summary, the eigenvectors provide the "skeleton" or grid upon which the dynamics occur, while the eigenvalues determine the speed and direction (inward or outward) of the movement along that skeleton.

The phase portrait confirms this behavior, showing trajectories approaching the direction of the dominant eigenvector while moving away from the origin.

3.3 Application 3: Quadratic Forms and Rotation of Axes

3.3.1 Quadratic Equation

$$ax^2 + bxy + cy^2 + dx + ey + f = 0$$

3.3.2 Matrix Form

Quadratic part:

$$ax^2 + bxy + cy^2 = X^T AX$$

where:

$$A = \begin{pmatrix} a & b/2 \\ b/2 & c \end{pmatrix}$$

Note: b is divided by 2 to correctly reproduce xy term.

The Principal Axes Theorem

The **Principal Axes Theorem** states that for every quadratic form $Q(\mathbf{x}) = \mathbf{x}^T A\mathbf{x}$, where A is a symmetric $n \times n$ matrix, there exists an orthogonal change of variable $\mathbf{x} = P\mathbf{y}$ such that:

$$Q(\mathbf{x}) = \mathbf{y}^T D\mathbf{y} = \lambda_1 y_1^2 + \lambda_2 y_2^2 + \cdots + \lambda_n y_n^2$$

where $\lambda_1, \ldots, \lambda_n$ are the eigenvalues of A, and the columns of the orthogonal matrix P are the corresponding orthonormal eigenvectors of A.

Geometrically, this means that any quadratic curve (such as an ellipse) can be rotated into a standard position where its axes align with the coordinate axes. In this "principal" coordinate system, the cross-product terms (e.g., xy) are eliminated, and the new coefficients are exactly the eigenvalues of the original matrix.

3.3.3 Example

$$13x^2 - 10xy + 13y^2 - 72 = 0$$

Matrix:

$$A = \begin{pmatrix} 13 & -5 \\ -5 & 13 \end{pmatrix}$$

Eigenvalues:

$$\lambda = 18, 8$$

Eigenvectors:

$$v_1 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \quad v_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

3.3.4 Form Matrix P

$$P = \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix}$$

3.3.5 Change of Variables

$$X = PX'$$

This rotates the coordinate system.

We define the orthogonal transformation as:

$$\mathbf{x} = P\mathbf{x}'$$

where $\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$ are the original coordinates and $\mathbf{x}' = \begin{bmatrix} x' \\ y' \end{bmatrix}$ are the principal (rotated) coordinates.

Substituting this into the quadratic form $\mathbf{x}^T A\mathbf{x} = 72$ gives:

$$(P\mathbf{x}')^T A(P\mathbf{x}') = 72$$

$$(\mathbf{x}')^T (P^T AP)\mathbf{x}' = 72$$

Since $P^T AP = D$ (the diagonal matrix of eigenvalues), the equation becomes:

$$\lambda_1 (x')^2 + \lambda_2 (y')^2 = 72$$

3.3.6 Final Equation

$$18x'^2 + 8y'^2 = 72$$

$$\frac{x'^2}{4} + \frac{y'^2}{9} = 1$$

Result: Ellipse

3.3.7 Important Concepts

- Eigenvectors → new axes (rotation)
- Eigenvalues → stretching along axes
- xy term → indicates rotation
- Linear terms $(dx + ey + f)$ → shift position only

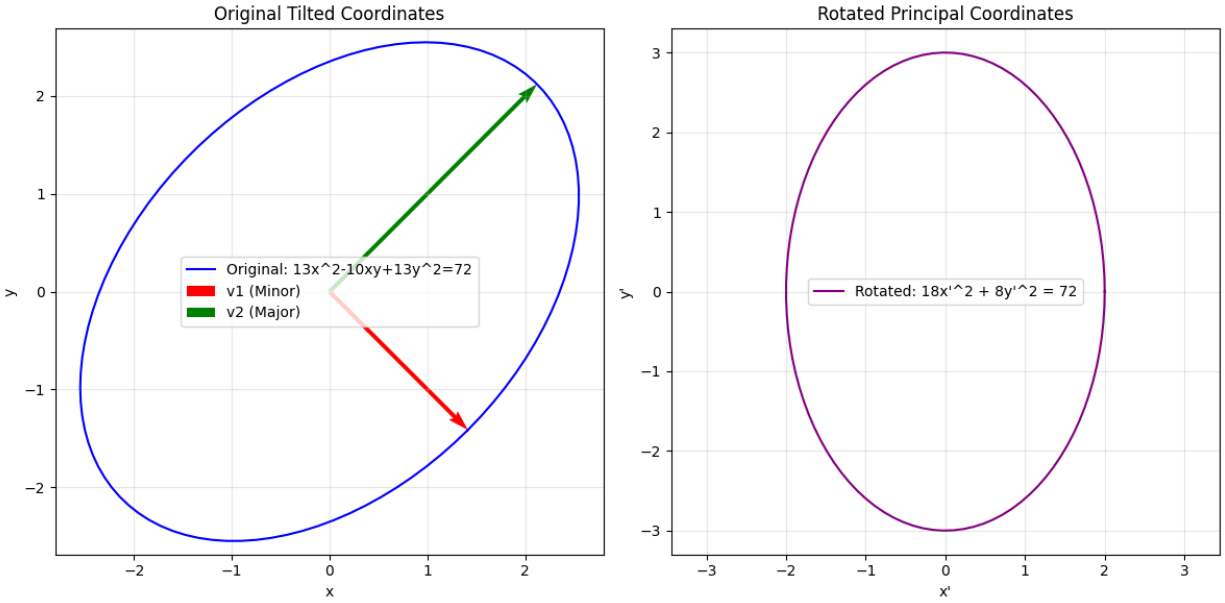
```
import numpy as np
import matplotlib.pyplot as plt

# 1. Construct the symmetric matrix A
# For 13x^2 - 10xy + 13y^2, A = [[a, b/2], [b/2, c]]
A = np.array([[13, -5],
              [-5, 13]])

# 2. Compute the eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(A)
lam1, lam2 = eigenvalues

print("Eigen-Analysis ---")
print(f"Eigenvalues (λ): {lam1:.2f}, {lam2:.2f}")
print(f"Eigenvectors:\n P={eigenvectors}")

# 3. Form orthogonal matrix P and verify P^T * A * P = D
P = eigenvectors # np.linalg.eig returns normalized eigenvectors
D = P.T @ A @ P

print("\n Verification (P^T * A * P) ---")
print(D.round(2))

# 5 & 6. Rewrite and identify the conic
# 18x'^2 + 8y'^2 = 72 => x'^2/4 + y'^2/9 = 1
print("\n Standard Form ---")
print(f"Equation: {lam1:.0f}x'^2 + {lam2:.0f}y'^2 = 72")
print(f"Standard Form: x'^2/{72/lam1:.0f} + y'^2/{72/lam2:.0f} = 1")
print("The conic is an Ellipse.")

# 7. Plotting the Conics
theta = np.linspace(0, 2*np.pi, 100)

# Rotated coordinates (Standard Ellipse)
a_prime = np.sqrt(72/lam1)
b_prime = np.sqrt(72/lam2)
x_prime = a_prime * np.cos(theta)
y_prime = b_prime * np.sin(theta)
```

```
# Original coordinates (using X = P * X_prime)
original_coords = P @ np.vstack([x_prime, y_prime])
x_orig = original_coords[0, :]
y_orig = original_coords[1, :]

plt.figure(figsize=(12, 6))

# Subplot 1: Original (Tilted) Conic
plt.subplot(1, 2, 1)
plt.plot(x_orig, y_orig, 'b-', label='Original: 13x^2-10xy+13y^2=72')

# Plot Principal Axes (Eigenvectors)
plt.quiver(0, 0, P[0,0]*a_prime, P[1,0]*a_prime, color='r', scale=1,
    scale_units='xy', label='v1 (Minor)')
plt.quiver(0, 0, P[0,1]*b_prime, P[1,1]*b_prime, color='g', scale=1,
    scale_units='xy', label='v2 (Major)')
plt.title('Original Tilted Coordinates')
plt.xlabel('x')
plt.ylabel('y')
plt.axis('equal')
plt.grid(True, alpha=0.3)
plt.legend()

# Subplot 2: Rotated (Standard) Conic
plt.subplot(1, 2, 2)
plt.plot(x_prime, y_prime, 'purple', label="Rotated: 18x'^2 + 8y'^2 = 72")
plt.title('Rotated Principal Coordinates')
plt.xlabel("x'")
plt.ylabel("y'")
plt.axis('equal')
plt.grid(True, alpha=0.3)
plt.legend()

plt.tight_layout()
plt.show()
```

OUTPUT

```
Eigen-Analysis ---
Eigenvalues (λ): 18.00, 8.00
Eigenvectors:
 P=[[ 0.70710678 0.70710678]
 [-0.70710678 0.70710678]]

 Verification (P^T * A * P) ---
[[18.  0.]
 [-0.  8.]]
```

```
 Standard Form ---
Equation: 18x'^2 + 8y'^2 = 72
Standard Form: x'^2/4 + y'^2/9 = 1
The conic is an Ellipse.
```



The process of diagonalizing the symmetric matrix associated with a quadratic form is physically and geometrically equivalent to a rotation of the coordinate system. Based on the numerical output and the generated plots, we observe the following:

The orthogonal matrix P, composed of normalized eigenvectors, represents a **rotation of axes**. For the given matrix $A = \begin{pmatrix} 13 & -5 \\ -5 & 13 \end{pmatrix}$,

- The transformation $X = PX'$ maps the "tilted" coordinates (x, y) to the "principal" coordinates (x', y').
- Because P is orthogonal ($P^T = P^{-1}$), the transformation preserves the distance and angles, meaning the shape of the conic is not distorted, only its orientation is changed.

The cross-term $-10xy$ in the original equation $13x^2 - 10xy + 13y^2 = 72$ represents a coupling between the x and y dimensions. By applying the transformation $X'^T DX'$, we obtain:

$$18x'^2 + 8y'^2 = 72 \implies \frac{x'^2}{4} + \frac{y'^2}{9} = 1 \tag{12}$$

The vanishing of the xy-term (visually confirmed by the alignment of the ellipse in the second plot) proves that the new axes x' and y' are the **principal axes** of the conic.

The "eigen-decomposition" reveals the hidden structure of the geometric shape:

- **Eigenvectors (Principal Axes):** The eigenvectors determine the *direction* of the major and minor axes. In the first plot, the red and green vectors point exactly along the longest and shortest diameters of the ellipse.

  When calculating the eigenvectors of the symmetric matrix $A = \begin{pmatrix} 13 & -5 \\ -5 & 13 \end{pmatrix}$ manually, we find the basis for the eigenspaces by solving $(A - \lambda I)\mathbf{v} = 0$:

  - For $\lambda_1 = 18$, the manual eigenvector is $\mathbf{v}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$.
  - For $\lambda_2 = 8$, the manual eigenvector is $\mathbf{v}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

  However, the Python `numpy.linalg.eig` function returns **normalized eigenvectors** (unit vectors with a magnitude of 1). To normalize the manual vectors, we divide each by its magnitude $\|\mathbf{v}\| = \sqrt{1^2 + 1^2} = \sqrt{2}$:

  $$\mathbf{v}_{normalized} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ \pm 1 \end{bmatrix} \approx \begin{bmatrix} 0.7071 \\ \pm 0.7071 \end{bmatrix}$$

  This explains why the matrix P obtained in the code consists of the values $\approx \pm 0.7071$.

  The orthogonal matrix P obtained from the Python output is:

  $$P = \begin{bmatrix} 0.7071 & 0.7071 \\ -0.7071 & 0.7071 \end{bmatrix}$$

  This matrix satisfies the condition $\det(P) = 1$, confirming it is a **proper rotation matrix**. By comparing this to the general rotation matrix R_θ:

  $$R_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

  we observe that for our specific matrix P, the value of $\sin\theta$ must be -0.7071. This implies that:

  $$\theta = -45^\circ \quad (\text{or } 45^\circ \text{ clockwise})$$

  This clockwise rotation is geometrically necessary to align the original coordinate axes with the principal axes of the ellipse (the eigenvectors). Specifically, it maps the original x-axis toward the eigenvector $\mathbf{v}_1 = [1, -1]^T$, which corresponds to the minor axis of the ellipse.

- **Eigenvalues (Shape and Scale):** The eigenvalues determine the *magnitude* of the stretch along those axes. Specifically, the lengths of the semi-axes are given by $\sqrt{k/\lambda_i}$ (where $k = 72$). Thus, the smaller eigenvalue ($\lambda = 8$) corresponds to the longer axis (3 units), and the larger eigenvalue ($\lambda = 18$) corresponds to the shorter axis (2 units).

The Principal Axes Theorem allows us to identify the conic as an **ellipse**. By choosing a basis of eigenvectors, we move from a coordinate system where the variables are "tangled" to one where the geometry of the object is perfectly aligned with the axes.

3.4 Unified View

All three applications rely on the same idea:

- Eigenvectors give natural directions
- Eigenvalues describe behavior along those directions